Un projet Python professionnel

Environnements virtuels, pip, requirements.txt, arguments en ligne de commande et chemins robustes. C'est ce qui fait la différence entre « ça marche sur mon ordinateur » et « ça marche sur n'importe quel ordinateur ». Appliqué directement à ExamPredict AI.

1. Le problème : « chez moi ça marche »
2. Les bibliothèques et pip
3. L'environnement virtuel (venv)
4. requirements.txt
5. La structure d'un vrai projet
6. Donner des ordres à un script : sys.argv
7. Des chemins qui marchent partout : pathlib
8. Les erreurs d'installation
9. Mission : ExamPredict installable
10. Mini-test corrigé
11. À savoir par cœur

---

1. Le problème : « chez moi ça marche »

Tu envoies ExamPredict à un recruteur. Il tape `python src/main.py` et obtient :

```
ModuleNotFoundError: No module named 'pypdf'
```

Sur ton ordinateur, `pypdf` est installé. Sur le sien, non. Et il ne sait pas quelle version installer. **Ton projet est inutilisable**, et c'est la première impression qu'il garde de toi.

> **Dans le métier**
>
> Un modèle d'IA entraîné avec une version de bibliothèque peut donner des résultats différents, ou planter, avec une autre version. La **reproductibilité** (pouvoir refaire exactement la même chose ailleurs) est une exigence de base en ML. Cette fiche t'apprend à la garantir.

2. Les bibliothèques et `pip`

> **Définition 2.1**
>
> Une **bibliothèque** (ou *package*) est du code écrit par d'autres, qu'on réutilise : `pypdf` lit les PDF, `numpy` calcule, `scikit-learn` entraîne des modèles. **pip** est le programme qui les télécharge et les installe depuis PyPI, le grand magasin des bibliothèques Python.
>
> **En clair :** pip, c'est l'App Store de Python.

COMMANDE	EFFET
`python -m pip install pypdf`	installe une bibliothèque
`python -m pip install pypdf==6.1.0`	installe une version précise
`python -m pip list`	liste ce qui est installé
`python -m pip show pypdf`	détails d'une bibliothèque (version, dossier)
`python -m pip uninstall pypdf`	désinstalle

On écrit `python -m pip` plutôt que `pip` tout court : on est ainsi sûr d'utiliser le pip **du Python qu'on vise**.

3. L'environnement virtuel (`venv`)

Analogie : sans venv, tous tes projets partagent **une seule cuisine**. Le projet A veut une casserole en version 1, le projet B la veut en version 2 : dispute. Avec venv, **chaque projet a sa propre cuisine**, avec ses propres ustensiles.

> **Définition 3.1**
>
> Un **environnement virtuel** est un dossier (souvent `.venv`) qui contient un Python et des bibliothèques **propres à un seul projet**, isolés du reste de l'ordinateur.

Le cycle de vie, sur Windows avec Git Bash

```
cd ~/AI-ENGINEER-2027/exampredict-ai
py -m venv .venv                    # 1. CRÉER la cuisine (une seule fois)
source .venv/Scripts/activate       # 2. ENTRER dans la cuisine (à chaque nouvell
                                    #    → (.venv) apparaît au début de la ligne
which python                        # 3. VÉRIFIER : doit finir par .venv/Scripts/
python -m pip install pypdf         # 4. INSTALLER, dans cette cuisine uniquement
deactivate                          # 5. SORTIR de la cuisine
```

> **Bonne nouvelle**
>
> Une fois le venv activé, la commande `python` marche enfin : c'est le Python du venv qui répond, et le faux raccourci du Microsoft Store disparaît.

> **Les 3 pièges**
>
> 1. **Oublier d'activer** : on installe sans le savoir dans le Python global. Regarde toujours si `(.venv)` est affiché.
> 2. **Mettre `.venv` sur GitHub** : il pèse des centaines de mégaoctets et ne marche que sur ta machine. Il va dans le `.gitignore`.
> 3. Sur Mac et Linux, le chemin est `.venv/bin/activate` au lieu de `.venv/Scripts/activate`.

---

4. `requirements.txt`

> **Définition 4.1**
>
> `requirements.txt` est la **liste de courses** du projet : une bibliothèque par ligne, avec sa version.

```
# requirements.txt
pypdf==6.1.0
pytest==8.3.3
```

COMMANDE	RÔLE
`python -m pip freeze > requirements.txt`	**Écrire** la liste : tout ce qui est installé dans le venv, avec les versions exactes
`python -m pip install -r requirements.txt`	**Lire** la liste : installer tout d'un coup sur un nouvel ordinateur

Remarque le `>` de la fiche F1.DAY4 : `pip freeze` affiche la liste, et `>` l'écrit dans le fichier.

> **La procédure que suivra le recruteur (à mettre dans le README)**
>
> ```
> git clone https://github.com/joe-web-startup/exampredict-ai.git
> cd exampredict-ai
> py -m venv .venv
> source .venv/Scripts/activate
> python -m pip install -r requirements.txt
> python src/main.py
> ```

5. La structure d'un vrai projet

```
exampredict-ai/
├── .venv/              # la cuisine privée       → dans .gitignore
├── data/               # les données             → souvent dans .gitignore
├── src/                # le code du produit
├── tests/              # les tests automatiques
├── .gitignore          # ce que Git doit ignorer
├── requirements.txt    # la liste de courses
├── pytest.ini          # les réglages des tests
└── README.md           # la vitrine : quoi, comment installer, comment lancer
```

6. Donner des ordres à un script : `sys.argv`

Comme `$1` en Bash, un script Python peut recevoir des mots tapés après son nom.

```python
# stats.py
import sys

print(sys.argv)                # la liste de tous les mots de la commande
fichier = sys.argv[1]          # le 1er mot après le nom du script
print(f"J'analyse {fichier}")
```

```
python stats.py notes.txt
# ['stats.py', 'notes.txt']
# J'analyse notes.txt
```

Piège : l'argument oublié

Si on lance `python stats.py` sans rien après, `sys.argv[1]` n'existe pas : `IndexError`. Un script pro vérifie d'abord :

```python
if len(sys.argv) < 2:
    print("Usage : python stats.py FICHIER")
    sys.exit(1)                # on quitte proprement, avec un code d'erreu
```

Dans le métier

On lance les entraînements comme ça : `python train.py --epochs 10 --lr 0.001`. On change les réglages sans toucher au code. La bibliothèque standard `argparse` gère ces options nommées.

---

7. Des chemins qui marchent partout : `pathlib`

Souviens-toi du bug : `data/exams` introuvable quand on lance depuis `src/`. La solution pro, c'est de construire le chemin **à partir de l'emplacement du fichier Python lui-même**, et non à partir de l'endroit où l'on tape la commande.

```
from pathlib import Path

ICI = Path(__file__).parent              # le dossier qui contient CE fichier :
RACINE = ICI.parent                      # le dossier du dessus : exampredict-ai
EXAMS = RACINE / "data" / "exams"        # le / de pathlib assemble les morceaux

print(EXAMS.exists())                    # True, quel que soit l'endroit d'où on
for pdf in EXAMS.glob("*.pdf"):          # comme find -name "*.pdf"
    print(pdf.name)
```

En clair : `__file__` veut dire « moi-même ». Au lieu de dire « la deuxième porte à gauche depuis là où tu es », le programme dit « la deuxième porte à gauche depuis **chez moi** ». Ça marche toujours.

8. Les erreurs d'installation

MESSAGE	CAUSE	REMÈDE
`ModuleNotFoundError: No module named 'x'`	la bibliothèque n'est pas installée **dans ce Python**	activer le venv, puis `python -m pip install x`
`Python was not found`	faux raccourci Windows, ou Python absent du `PATH`	`py`, ou activer le venv
`command not found`	programme inconnu du terminal	vérifier l'orthographe, puis le `PATH`
`FileNotFoundError`	chemin relatif lancé depuis le mauvais dossier	`pwd`, ou `pathlib` + `__file__`
`IndexError: list index out of range` sur `sys.argv`	argument oublié	vérifier `len(sys.argv)`

9. Mission : ExamPredict installable

À faire au terminal (Git Bash), étape par étape (45 min)

1. Va dans `~/AI-ENGINEER-2027/exampredict-ai`.
2. Crée le venv `.venv` et active-le. Vérifie `(.venv)` et `which python`.
3. Installe `pypdf` et `pytest` avec `python -m pip`.
4. Écris `requirements.txt` avec `pip freeze >`, puis affiche-le avec `cat`.
5. Ajoute `.venv/` au `.gitignore` (`echo ".venv/" >> .gitignore`).
6. Lance `python src/main.py` depuis la racine : ça marche. Puis `cd src` et relance `python main.py` : observe la `FileNotFoundError`.
7. Corrige `main.py` avec `pathlib` et `__file__` (section 7), pour que le programme marche **des deux endroits**.
8. Branche enfin `count_keyword` à la ligne 149 d'`analyzer.py` et compare le classement avant et après.
9. Teste l'installation « comme un recruteur » : `deactivate`, supprime `.venv`, recrée-le, réinstalle avec `pip install -r requirements.txt`, relance. Tout doit remarcher.

10. Mini-test corrigé

1. À quoi sert un environnement virtuel ? ★

À donner à chaque projet ses propres bibliothèques et versions, isolées des autres projets. Une cuisine par projet.

2. Quelle commande écrit la liste des bibliothèques installées dans requirements.txt ? ★

```
python -m pip freeze > requirements.txt
```

3. Tu vois `ModuleNotFoundError: No module named 'pypdf'`. Quelles sont les deux premières choses à vérifier ? ★★

1. Le venv est-il activé (`(.venv)` affiché) ? 2. pypdf est-il installé dedans (`python -m pip show pypdf`) ?

4. Pourquoi `.venv` ne va-t-il jamais sur GitHub ? ★★

Il est très lourd et ne fonctionne que sur ta machine. On partage la **liste de courses** (`requirements.txt`), pas la cuisine.

5. Que contient `sys.argv` pour la commande `python stats.py notes.txt 20` ? ★★★

`['stats.py', 'notes.txt', '20']` . Attention : `'20'` est un **texte**. Il faut `int(sys.argv[2])` pour calculer avec.

11. À savoir par cœur

1. Une bibliothèque, c'est du code réutilisé. `pip` , c'est l'App Store de Python. On écrit toujours `python -m pip` .
2. venv = une cuisine par projet. `py -m venv .venv` , puis `source .venv/Scripts/activate` , puis `deactivate` .
3. Toujours vérifier que `(.venv)` est affiché avant d'installer.
4. `pip freeze > requirements.txt` écrit la liste. `pip install -r requirements.txt` la lit.
5. `.venv/` va dans le `.gitignore` . On partage la liste de courses, pas la cuisine.
6. `sys.argv` est la liste des mots de la commande. `sys.argv[1]` est le 1er argument, et c'est du texte.
7. `Path(__file__).parent` donne des chemins qui marchent d'où que l'on lance le programme.
8. `ModuleNotFoundError` : vérifier le venv, puis l'installation.
